

AtCoder ARC127 – A

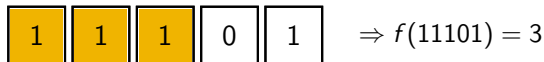
Leading 1s の解説

AtCoder Regular Contest 127 / 300 点

問題の概要

整数 x に対し, $f(x) = x$ を十進表記したときの先頭から連続する 1 の個数と定める.

連続する 1 が 3 個



ここで途切れる

他の例: $f(1) = 1$, $f(2) = 0$, $f(10) = 1$, $f(11) = 2$, $f(101) = 1$

求めるもの: $f(1) + f(2) + \dots + f(N)$

制約

$1 \leq N \leq 10^{15}$ (高々 16 桁) $\Rightarrow$ 1 個ずつ f を計算する $O(N)$ は論外. 数え方を変える必要がある.

核心 - f を 0/1 判定の和にバラす

$f(x)$ をそのまま計算しようとする x ごとのループになって詰む。そこで

$$f(x) = \sum_{k \geq 1} [x \text{ の先頭 } k \text{ 桁がすべて } 1]$$

($[\cdot]$ は成り立てば 1, そうでなければ 0)

k	$x = 111$ の先頭 k 桁	判定
1	1	真
2	11	真
3	111	真
4	(桁が足りない)	偽

真になる k はちょうど 3 個 $= f(111)$

※ 同じ x を複数回数えるのは仕様。

それがちょうど $f(x)$ になる。

和の順序を入れ替えると

$$\text{答え} = \sum_{k \geq 1} \#\{x \leq N : \text{先頭 } k \text{ 桁が全部 } 1\}$$

何が起きたか

x のループ (10^{15} 回) が

k のループ (高々 16 回) に化けた。

先頭が固定された数の集合 = 区間の集まり

$\text{head} = \underbrace{11 \cdots 1}_{k \text{ 個}}$ とおく. 「先頭 k 桁が 1 で, 後ろに i 桁続く数」は, $L = \text{head} \times 10^i$ として

$$[L, L + 10^i - 1] \text{ のちょうど } 10^i \text{ 個}$$

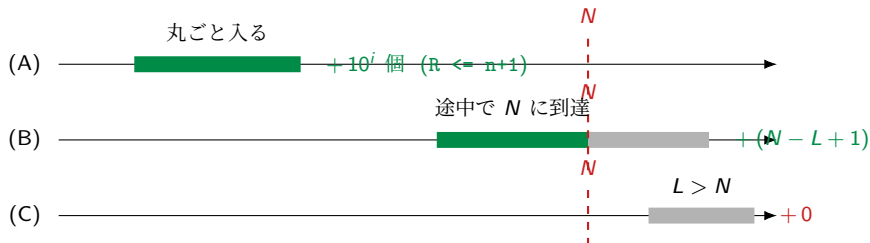
例: $k = 3$ (head = 111)



桁数が違うので, i が違う区間どうしは**重ならない**. よって各 k について「区間の個数を足すだけ」でよい.

N で打ち切る - 3 つのケース

区間 $[L, R - 1]$ ($R = L + 10^i$) に対し, N 以下の部分だけを数える.



(B)(C) はどちらも以降の i でも L が 10 倍ずつ大きくなるので, そこで break してよい.

擬似コード

```
ans = 0 ; head = 0
for k in 1..16:                                # N <= 10^15 は高々 16 桁
    head = head * 10 + 1                        # 1, 11, 111, ...
    L = head
    for i in 0..15:
        if L > N: break                        # (C) この先はすべて範囲外
        R = L + 10^i                           # 区間は [L, R-1]
        if R <= N + 1:
            ans += R - L                        # (A) 丸ごと入る = 10^i 個
        else:
            ans += N - L + 1                    # (B) 途中まで
            break
    L *= 10
```

計算量 $O(D^2) \approx 16 \times 16 = 256$ 回 ($D =$ 桁数). N の大きさに依存しない.

サンプル 2 ($N = 120$, 答え 44) を手で追う

k	head	i	L	区間 $[L, R - 1]$	判定	加算
1	1	0	1	$[1, 1]$	(A) 丸ごと	+1
1	1	1	10	$[10, 19]$	(A) 丸ごと	+10
1	1	2	100	$[100, 199]$	(B) 120 で打ち切り	+21
2	11	0	11	$[11, 11]$	(A) 丸ごと	+1
2	11	1	110	$[110, 119]$	(A) 丸ごと	+10
2	11	2	1100	—	(C) $L > 120$	break
3	111	0	111	$[111, 111]$	(A) 丸ごと	+1
3	111	1	1110	—	(C) $L > 120$	break
4	1111	0	1111	—	(C) $L > 120$	break

小計は $k = 1$: **32**, $k = 2$: **11**, $k = 3$: **1** $\Rightarrow$ 合計 $32 + 11 + 1 = \mathbf{44}$ ✓

それぞれ「先頭が 1 の数の個数」「先頭が 11 の数の個数」「先頭が 111 の数の個数」.

落とし穴 – if $L > n$: break を省くと静かに壊れる

$L > N$ のとき, (B) の枝が実行されると

$$N - L + 1 \leq 0$$

すなわち負の数を足してしまう.

引き算で個数を出すコードは, 範囲外ガードを忘れると例外も出さずに答えだけが壊れる. 個数を出す式を書いたら「この式が負になる入力はあるか」を必ず確認する.

その他の注意点:

- $R \leq n + 1$ は $R - 1 \leq n$ の言い換え (区間の右端が N 以下).
- Python の int は多倍長なので溢れない. C++ 移植時は long long (N も L も 10^{15} 規模で, int では即オーバーフロー).
- ループ上限 16 は $N \leq 10^{15}$ が 16 桁だから. 制約が 10^{18} なら 19 に増やす.

実測 (ガードを外した版)

- $N = 1$ の出力:
-1234567901234542 (正解 1)
- $1 \leq N \leq 500$ の
500 通りすべてが不正解

まとめ

- ① $\sum f(x)$ が重いときは f を 0/1 判定の和に分解し、和の順序を入れ替える（寄与の分解）.

$$\sum_x f(x) = \sum_x \sum_k [\text{条件}_k(x)] = \sum_k \#\{x : \text{条件}_k\}$$

x のループが「条件のループ」に化けて、 N に依存しない計算量になる.

- ② 接頭辞が固定された数の集合 = 区間の和. $[\text{head} \cdot 10^i, \text{head} \cdot 10^i + 10^i - 1]$ を i について足す. 桁 DP を持ち出すのは過剰.
- ③ 区間は半開 $[L, R)$ で持つと個数が $R - L$ になり, off-by-one が減る.
- ④ 「範囲外なら break」のガードを省かない. 引き算で個数を出すコードは負の数を足して静かに壊れる.

同系統の問題: ABC154 E – Almost Everywhere Zero, ABC029 D – 1（桁ごとの寄与）, ABC192 D – Base n